ASSIGNMENT – 12

NAME : SATHWIK REDDY

HALL TICKET NO : 2303A51989

BATCH : 15

Question :

Hyperledger Fabric Problem Solving Objective To design and implement a simple Hyperledger Fabric smart contract (chaincode) that manages a personal asset portfolio on a blockchain network. The practical focuses on problem-solving using Fabric concepts such as assets, transactions, ledger state, and queries. Problem Statement Implement a basic blockchain-based portfolio system where a user can: • Create assets (e.g., certificates, projects, or tokens) • View all assets stored on the blockchain • Transfer asset ownership between users This practical demonstrates how Hyperledger Fabric solves realworld problems like data immutability, transparency, and secure ownership transfer. Requirements Software Requirements • Operating System: Windows / Linux / macOS • Node.js (v16 or later) • Docker & Docker Compose • Git • VS Code (with Docker and JavaScript extensions) Blockchain Platform • Hyperledger Fabric test network

CODE :

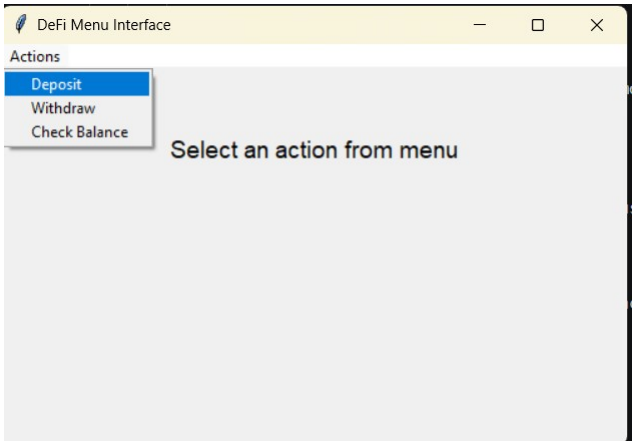
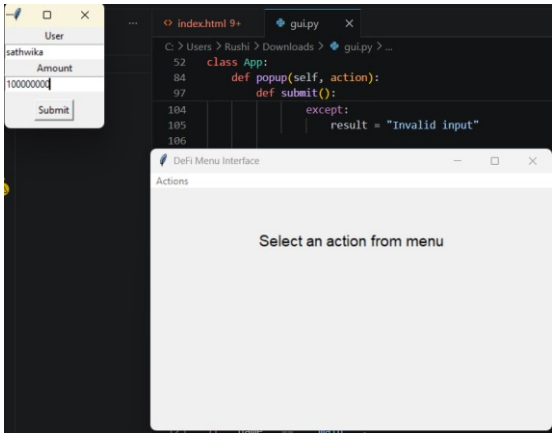
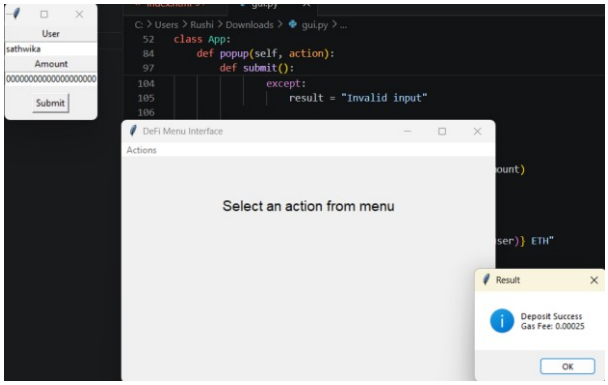
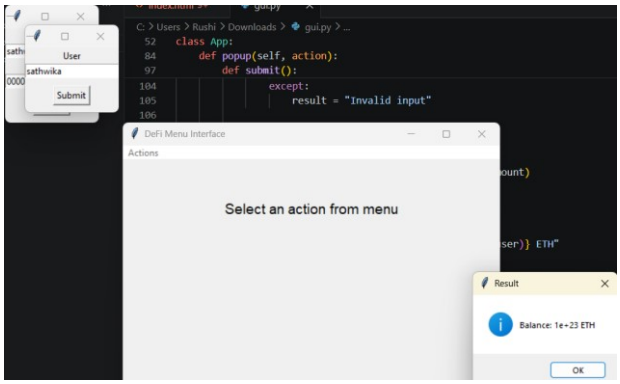
```
import tkinter as tk from tkinter import messagebox

import random import time class DeFi:     def

__init__(self):        self.balances = {}     def gas(self):

return round(random.uniform(0.0001, 0.001), 6)

def delay(self):        time.sleep(0.3)     def

deposit(self, user, amount):        if amount <= 0:

            return "Invalid amount"

self.delay()        gas = self.gas()     if

user not in self.balances:

self.balances[user] = 0

self.balances[user] += amount

return f"Deposit Success\nGas

Fee: {gas}"     def withdraw(self,

user, amount):        if user not in

self.balances:            return "User

not found"        if

self.balances[user] < amount:

return "Insufficient balance"

self.delay()        gas = self.gas()
```

```
self.balances[user] -= amount
return f"Withdraw Success\nGas
Fee: {gas}"     def balance(self,
user):
        return self.balances.get(user,
0) class App:     def __init__(self,
root):
        self.defi = DeFi()        self.root = root        self.root.title("DeFi Menu Interface")
self.root.geometry("500x300")        menubar = tk.Menu(root)
root.config(menu=menubar)        action_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="Actions", menu=action_menu)
action_menu.add_command(label="Deposit", command=self.deposit_window)
action_menu.add_command(label="Withdraw", command=self.withdraw_window)
action_menu.add_command(label="Check Balance", command=self.balance_window)
self.label = tk.Label(root, text="Select an action from menu",
                        font=("Arial", 14))
self.label.pack(pady=50)     def
deposit_window(self):
self.popup("Deposit") def
withdraw_window(self):
self.popup("Withdraw") def
balance_window(self):
self.popup("Balance")     def
popup(self, action):        win =
tk.Toplevel(self.root)
win.title(action)        tk.Label(win,
text="User").pack()
user_entry = tk.Entry(win)
user_entry.pack()        if action !=
"Balance":
            tk.Label(win,
text="Amount").pack()
```

```
amount_entry = tk.Entry(win)
amount_entry.pack()        def submit():
        user = user_entry.get()
if action == "Deposit":
            try:
                amount = float(amount_entry.get())
result = self.defi.deposit(user, amount)
except:
                result = "Invalid input"
elif action == "Withdraw":
            try:
                amount = float(amount_entry.get())
result = self.defi.withdraw(user, amount)
            except:
                result = "Invalid input"
else:
            result = f"Balance: {self.defi.balance(user)} ETH"
messagebox.showinfo("Result", result)        tk.Button(win,
text="Submit", command=submit).pack(pady=10) if __name__ ==
"__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()
```

IMPLEMENTATION :

```python
import tkinter as tk
from tkinter import messagebox
import random
import time
class DeFi:
    def __init__(self):
        self.balances = {}

    def gas(self):
        return round(random.uniform(0.0001, 0.001), 6)

    def delay(self):
        time.sleep(0.3)

    def deposit(self, user, amount):
        if amount <= 0:
            return "Invalid amount"

        self.delay()
        gas = self.gas()

        if user not in self.balances:
            self.balances[user] = 0

        self.balances[user] += amount
        return f"Deposit Success\nGas Fee: {gas}"

    def withdraw(self, user, amount):
        if user not in self.balances:
            return "User not found"

        if self.balances[user] < amount:
            return "Insufficient balance"
```

```python
class App:
    def popup(self, action):
        user_entry = tk.Entry(win)
        user_entry.pack()

        if action != "Balance":
            tk.Label(win, text="Amount").pack()
            amount_entry = tk.Entry(win)
            amount_entry.pack()

        def submit():
            user = user_entry.get()

            if action == "Deposit":
                try:
                    amount = float(amount_entry.get())
                    result = self.defi.deposit(user, amount)
                except:
                    result = "Invalid input"
            elif action == "Withdraw":
                try:
                    amount = float(amount_entry.get())
                    result = self.defi.withdraw(user, amount)
                except:
                    result = "Invalid input"
            else:
                result = f"Balance: {self.defi.balance(user)} ETH"
            messagebox.showinfo("Result", result)
        tk.Button(win, text="Submit", command=submit).pack(pady=10)
if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()
```

OUTPUT :